

Program — Production-Readiness Audit

A code-verified assessment of the MERN e-learning MVP — every item below was checked against the current `main` branch, not assumed from docs or commit messages.

Repo program-merged-update **Ref** main @ eaa5950 **Date** 26 Jul 2026
Method direct code read + live testing + 3 independent verification passes



Bottom line: the three-role backend and most primary frontend flows are genuinely wired and correctly authorized — that's a real majority of the feature surface, verified route-by-route. But a crash-level bug currently blocks the single most core student action (watching a lesson), the media upload pipeline is unverified/broken in the only environment tested, a fully-built fake payment UI contradicts written scope, and there is zero deployment infrastructure or CI. This is a mostly-complete app with one severe regression and a meaningful list of fake/half-built UI that needs a decision before launch — not a short polish pass.

01 Core Features — all three roles

Checked whether each flow calls a real backend endpoint end-to-end, or still uses mock/placeholder data anywhere in the frontend.

STUDENT

FLOW	STATUS	WHAT'S ACTUALLY TRUE IN THE CODE

Register (role selector)	● Done	Real role picker sends <code>role</code> in the payload; backend sanitizes it. <code>AuthPage.jsx:98,119–131</code>
Browse / search courses	● Done	Real <code>GET /courses</code> with debounced <code>search / category</code> params. <code>ExploreTab.jsx:31–37</code>
View course details	● Done	Real <code>GET /courses/:id</code> , no route-state-only fallback. <code>CoursePage.jsx:26–27</code>
Enroll	● Partially Done	Real <code>POST /enrollments/:id</code> fires, but it's wrapped in a fully-built fake payment UI (unwired buttons, "100% secure" copy) and shows a hardcoded mock user instead of the logged-in one. <code>CheckoutPage.jsx:3,48–105,229</code>
Watch video lessons	● Broken	Gated fetch is real, but the lesson list references an undefined <code>isActive</code> variable — throws and crashes the whole page for any course with lessons. <code>LearningPortal.jsx:473–474</code>
Mark lesson complete	● Done	Real <code>PATCH .../complete</code> , but unreachable in practice — blocked by the crash above. Also add-only, not a true toggle despite the name.
See progress	● Done	Computed server-side and returned as-is, not estimated client-side.

INSTRUCTOR

FLOW	STATUS	WHAT'S ACTUALLY TRUE IN THE CODE
Register as instructor	● Done	Same real path as student registration.
Dashboard	● Done	Real <code>GET /courses/mine</code> + <code>/courses/stats</code> , no mock arrays.
Create course	● Done	Real thumbnail upload then <code>POST /courses</code> with all real fields.
Add lesson + video upload	● Broken (this env)	Code path is real, but live-tested and confirmed failing: Cloudinary creds in <code>server/.env</code> are literal placeholders. Live call returned

Unknown API key `your_api_key`. Not verified working anywhere.

Submit for approval

● Done

By design — every new course is created as `pending`, no separate submit step.

ADMIN

FLOW	STATUS	WHAT'S ACTUALLY TRUE IN THE CODE
Login	● Done	Real <code>POST /auth/login</code> plus a client-side admin-role gate.
Dashboard stats	● Partially Done	Real aggregation endpoint, but the display layer has hardcoded fallback numbers that mask genuine zero-values with fake nonzero ones. <code>AdminOverviewTab.jsx:51-59</code>
View pending courses	● Done	Real <code>GET /courses/pending</code> , correctly role-gated.
Approve / reject	● Partially Done	Approve sets an invalid status value (<code>'published'</code> vs the real <code>approved</code> enum) — course vanishes from the list until refresh. Reject sends a hardcoded reason string , no real reason UI. <code>AdminCourseManagementTab.jsx:196,208</code>

02 Recent UI/UX Work

Each item from the last two rounds of UI fixes, checked individually against current code — not summarized as one block.

ITEM	STATUS	WHAT'S ACTUALLY TRUE IN THE CODE
Static pages / footer	● Done	Footer + all 6 static pages exist, routed, reachable. Privacy/Terms/MobileApp are intentional minimal placeholders, not broken.
Courses nav dropdown	● Done	Replaces old filter buttons; genuinely drives server-side category filtering.

Loading animations	● Done	Sitewide spinner/shimmer primitives, used across many unrelated components — not a one-off addition.
Admin Quick Actions removal	● Done	No functional references remain; only dead orphaned CSS left behind.
User Management consolidation	● Done	Single unified tab with a role sub-filter, no separate Student/Instructor components.
Course Management functionality	● Partially Done	No fake bulk-actions remain, but a new bug exists in the very commit that claims this fix — see Approve/Reject above.
Certificates removal	● Done	Exhaustive grep: zero functional references anywhere. Only stale doc mentions and one dead CSS class remain.
Financial Management cleanup	● Done	No duplicate "Enrollments" entry in either admin or superadmin sidebar.
Superadmin profile removal	● Done	Confirmed absent from the superadmin menu; correctly still present for regular admins.

03 Security

Read every route file and the core auth/crypto/sanitization middleware directly.

- **Cookies** — flags correctly conditioned on `NODE_ENV` (httpOnly always, secure/sameSite=none only in production). Code is production-correct; the local `.env` was misconfigured this session (fixed).
- **Password hashing** — bcrypt, cost 12, field excluded from queries by default. Correct.
- **JWT secret** — read from env, never hardcoded. Correct.
- **Authorization** — every route file checked; all protected routes correctly gated with `protect / authorize(...)`, and ownership checks are present on every instructor mutation reviewed. No unintentionally-open route found.
- **Video URL gating** — confirmed intact, not regressed. Public course endpoint excludes `videoUrl`; the gated lesson endpoint checks enrollment/ownership/admin before releasing it. One minor inconsistency: an `isAdmin` check on the course-details endpoint only tests `role === 'admin'`, not `superadmin`.
- **Rate limiting** — real and applied on login/register/check-email plus a global backstop. Two limiters (forgot-password, OTP) are defined but never wired to any route — dead scaffolding for an explicitly

out-of-scope feature.

- **Input sanitization** — Mongo-operator stripping and XSS escaping applied globally, layered with express-validator on auth routes. Solid.
- **CORS** — implementation is correct (env-driven allowlist), but no real production domain has ever been configured or tested, because nothing is deployed yet.
- **Security PRs** — Sprint 1 hardening (helmet, CSP, rate limiting, sanitization, CSRF double-submit, audit logging) is merged. This session I also fixed three unguarded null-dereference crashes and one IDOR (cross-course lesson overwrite) that were latent in that code. A remote branch `feature/enterprise-security(sprint2)` exists **unmerged** — if it contains intended fixes, they are not in `main`.

04 Bugs / Known Issues

No literal TODO/FIXME markers exist anywhere in the codebase — everything below was found by tracing UI-to-backend wiring directly, not by grepping comments.

Undefined `isActive` crashes the Learning Portal **CRITICAL**

Referenced at `LearningPortal.jsx:473-474` but never declared in scope — throws a `ReferenceError` on any course with lessons, taking down the core video-watching page.

Cloudinary uploads non-functional **CRITICAL**

Placeholder API credentials in the only environment available for testing — confirmed failing live with "Unknown API key `your_api_key`". Blocks the entire instructor content pipeline.

Approved courses vanish from the admin list **MODERATE**

Approve handler sets status to `'published'`, which matches neither the "Approved" nor "Pending" filter tab, until a manual refresh. `AdminCourseManagementTab.jsx:196`

Hardcoded mock identity shown at checkout **MODERATE**

`client/src/data.js`'s fake user (a real-looking name and email) is displayed to every user in CheckoutPage instead of the actual logged-in account.

Cluster of fake-success UI **MODERATE**

"Report issue" (console.log only), "Share" (never touches the clipboard), admin "Export CSV" (no export), the lesson Draft/Published toggle (no API call, no state update), and the backend "send test email" endpoint (setTimeout + hardcoded success) all tell the user something happened when nothing did.

System Management is mostly a demo shell MINOR

Storage, Backup, Logs, API keys, Feature Flags, AI, and Audit tabs are self-documented in a code comment as placeholder panels. Several toggles (2FA, phone verification, invite-only registration, auto-approve instructors, all feature flags) persist to the database but are read by nothing.

Build & lint MINOR

Build is clean, zero errors — one warning: main JS bundle is 1.04 MB (273 KB gzip), unsplit. Lint: zero errors, ~45 warnings, mostly unused vars and a whole parallel unwired admin-action implementation sitting dead next to the one actually in use.

05 Testing

- **Integration suite** — ran it directly, twice, this session: all assertions passed both times. Runs against an in-memory MongoDB, not the real Atlas database — matches what's documented, confirmed accurate.
- **Real Cloudinary upload** — **not verified working** anywhere. Tested live with the credentials available and it failed outright. No evidence either way for any other environment.
- **Manual QA** — substantial live testing happened this session incidentally (register/login/enroll/CORS/cookies/cascade-delete), but that's not a structured pass over every role's UI in a browser. No instructor or admin UI was clicked through this session. Treat full manual QA as **not confirmed done**.

06 Deployment Readiness

- **Not deployed anywhere persistent** — no GitHub Deployments, no Pages, no repo homepage, no Dockerfile/vercel.json/render.yaml/Procfile/netlify.toml found. The only external URL in play is a Cloudflare quick-tunnel address in a local `.env` — an ephemeral dev-sharing tool, not a real deployment.
- **No CI/CD** — no `.github/workflows` directory exists. Nothing runs tests or a build check automatically on PRs.

- `.env.example` — present and reasonably complete for both server and client, with minor drift: default `PORT` / `VITE_API_URL` reference port 5000, actual local dev value is 5050.
- **CORS/cookies for real prod domains** — code is written correctly to support this, but since nothing is deployed, it has never been exercised against an actual production domain.
- **Hardcoded dev-only data** — `client/src/data.js` mock data is still imported live in CheckoutPage; must be removed before launch.

07 Scope Check

Grepped the entire codebase, client and server, for every item on the documented out-of-scope list.

- **Payments / Stripe** — no real processor integration exists (zero references). But a fully-built **fake** payment UI is live and reachable — unwired payment-method buttons, "100% secure and encrypted / 30-day money-back guarantee" copy — that just enrolls for free underneath. Directly contradicts the written instruction to avoid fake payment simulation. A dead "Stripe Payments Integration" toggle in admin settings does nothing either way.
- **Live chat, gamification, wishlist, certificates** — confirmed absent. Only false-positive "badge" hits (unrelated status-pill UI) and stale doc mentions of certificates remain.
- **Mobile app** — correctly scoped as an honest "coming soon, use the mobile browser" static page; doesn't imply a real app exists.
- **Forgot password** — correctly not implemented as a working feature, but half-built scaffolding was left behind: a dead link with no `href` / `onClick`, plus unused rate limiters and matching config entries wired to nothing.

HALF-BUILT — NEEDS A DECISION, NOT A SHRUG

- CheckoutPage's fake payment UI — remove, replace with a direct enroll flow.
- System Management's placeholder tabs and dead toggles — hide/remove or finish; an admin could currently believe they enabled 2FA or backups when nothing happens.
- The fake-success UI cluster (Report issue, Share, Export CSV, lesson publish toggle, test-email endpoint) — wire for real or remove.
- Duplicate/dead admin reject-reason implementations — pick one, delete the other.
- Forgot-password dead scaffolding — delete.

– Prioritized Punch List

In order — this is a real sequence, each item assumes the ones above it are done.

- 1 **Fix the `isActive` crash** in `LearningPortal.jsx` — blocks core functionality, almost certainly a one-word fix. Do this first.
- 2 **Get real Cloudinary credentials into a real environment** and verify video/image/document upload end-to-end — currently unverified/broken, and blocks everything downstream of course creation.
- 3 **Fix the approve-course status bug** (`'published'` → `'approved'`) in `AdminCourseManagementTab.jsx`.
- 4 **Decide the fate of `CheckoutPage`** — replace with a plain enroll flow, or explicitly scope in real payments. Either way, stop shipping a fake payment UI with security/guarantee marketing copy.
- 5 **Remove the hardcoded mock identity** from `CheckoutPage` (`client/src/data.js`).
- 6 **Triage System Management's placeholder tabs** and dead toggles — hide/remove or finish; don't leave controls that silently do nothing.
- 7 **Fix or remove the fake-success UI cluster** — Report issue, Share, Export CSV, lesson publish toggle, test-email endpoint.
- 8 **Stand up an actual deployment** (staging at minimum) with real `CLIENT_URL` / `NODE_ENV` / `MONGO_URI` / Cloudinary values — nothing about production-readiness is fully confirmable until this exists.
- 9 **Add CI** — at minimum, run the integration suite and the client build/lint on every PR. Multiple redundant branches/PRs already show nothing currently stops a broken PR from merging.
- 10 **Clean up the rest** — dead forgot-password scaffolding, dead admin reject-reason code path, `.env.example` port drift, stale certificate doc mentions, bundle-size warning.

EXPLICITLY FLAGGED — COULD NOT VERIFY, NO ACCESS

- **Clouldinary in any other environment** — only this machine's `.env` was available, and it has placeholder credentials. No way to confirm or deny whether uploads have ever worked elsewhere with real credentials.
- **Any real production or staging environment** — none exists or was found. All CORS/cookie "correctness" above describes the code being written properly for when that environment exists, not evidence it's been tested against a real domain.
- **The unmerged `feature/enterprise-security(sprint2)` branch** — confirmed it exists, did not read its diff. If it holds intended fixes for this release, they are not currently in `main`.
- **Manual QA history** — no access to any QA log or tracker. Can only speak to what was personally tested this session, which is not a substitute for a full click-through of every role's UI.

Compiled by direct code inspection, live endpoint testing, and three independent parallel verification passes over the current `main` branch. File:line references point to state at commit eaa5950.